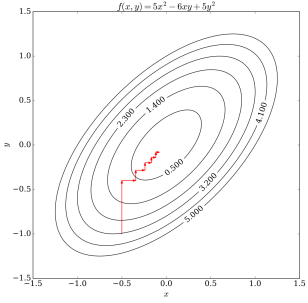
Optimization in Machine Learning

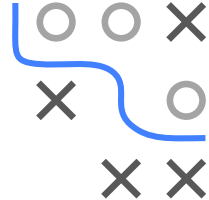
Coordinate descent



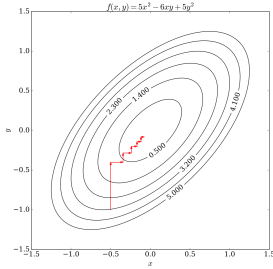
Learning goals

- Axes as descent direction
- CD on linear model and LASSO
- Soft thresholding

COORDINATE DESCENT



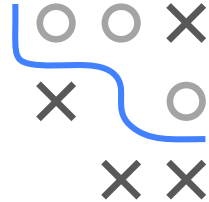
- **Assumption:** Objective function not differentiable
- **Idea:** Instead of gradient, use coordinate directions for descent
- First: Select starting point $\boldsymbol{x}^{[0]} = (x_1^{[0]}, \ldots, x_d^{[0]})$
- Step t: Minimize f along x_i for each dimension i for fixed $x_1^{[t]}, \ldots, x_{i-1}^{[t]}$ and $x_{i+1}^{[t-1]}, \ldots, x_d^{[t-1]}$:



Source: Wikipedia (Coordinate descent)

EXAMPLE: LINEAR REGRESSION

Minimize LM with L2-loss via CD:



$$\min g(\boldsymbol{\theta}) = \min_{\boldsymbol{\theta}} \frac{1}{2} \sum_{i=1}^{n} \left(y^{(i)} - \boldsymbol{\theta}^\top \mathbf{x}^{(i)} \right)^2 = \min_{\boldsymbol{\theta}} \frac{1}{2} \|\mathbf{y} - \mathbf{X}\boldsymbol{\theta}\|^2$$

where $\mathbf{y} \in \mathbb{R}^n$, $\mathbf{X} \in \mathbb{R}^{n \times d}$ with columns $\mathbf{x}_1, \ldots, \mathbf{x}_d \in \mathbb{R}^n$.

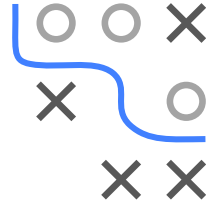
Assume: Scaled data, i.e., $\mathbf{X}^\top \mathbf{X} = I_d$ (just to get intuition)

Then:

$$\begin{aligned} g(\boldsymbol{\theta}) &= \frac{1}{2}\mathbf{y}^\top \mathbf{y} + \frac{1}{2}\boldsymbol{\theta}^\top \boldsymbol{\theta} - \mathbf{y}^\top \mathbf{X}\boldsymbol{\theta} \\ &\overset{(*)}{=} \frac{1}{2}\mathbf{y}^\top \mathbf{y} + \frac{1}{2}\boldsymbol{\theta}^\top \boldsymbol{\theta} - \mathbf{y}^\top \sum_{k=1}^{d} \mathbf{x}_k \theta_k \end{aligned}$$

$^{(*)}$ $\mathbf{X}\boldsymbol{\theta} = \mathbf{x}_1\theta_1 + \mathbf{x}_2\theta_2 + \cdots + \mathbf{x}_d\theta_d = \sum_{k=1}^{d} \mathbf{x}_k \theta_k$

SOFT THRESHOLDING



Minimize LM with L2-loss and L1 regularization via CD:

$$\min_{\boldsymbol{\theta}} h(\boldsymbol{\theta}) = \min_{\boldsymbol{\theta}} \frac{1}{2}\|\mathbf{y} - \mathbf{X}\boldsymbol{\theta}\|^2 + \lambda\|\boldsymbol{\theta}\|_1$$

Note that $h(\boldsymbol{\theta}) = \frac{1}{2}\mathbf{y}^\top\mathbf{y} + \frac{1}{2}\boldsymbol{\theta}^\top\boldsymbol{\theta} - \sum_{k=1}^{d}(\mathbf{y}^\top\mathbf{x}_k\theta_k + \lambda|\theta_k|)$

Assume (again): $\mathbf{X}^\top\mathbf{X} = I_d$.
Since $|\cdot|$ is not differentiable, distinguish three cases:

- **Case 1:** $\theta_j > 0$. Then $|\theta_j| = \theta_j$ and

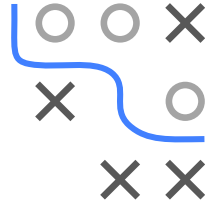
$$0 = \frac{\partial h(\boldsymbol{\theta})}{\partial \theta_j} = \theta_j - \mathbf{y}^\top\mathbf{x}_j + \lambda \qquad \Leftrightarrow \qquad \theta^*_{j,\text{LASSO}} = \theta^*_j - \lambda$$

- **Case 2:** $\theta_j < 0$. Then $|\theta_j| = -\theta_j$ and

$$0 = \frac{\partial h(\boldsymbol{\theta})}{\partial \theta_j} = \theta_j - \mathbf{y}^\top\mathbf{x}_j - \lambda \qquad \Leftrightarrow \qquad \theta^*_{j,\text{LASSO}} = \theta^*_j + \lambda$$

- **Case 3:** $\theta_j = 0$

CD FOR STATISTICS AND ML



Why is it being used?

- Easy to implement
- Scalable: no storage/operations on large objects, just current point
  $\Rightarrow$ Good implementation can achieve state-of-the-art performance
- Applicable for non-differentiable (but convex separable) objectives

Examples:

- Lasso regression, Lasso GLM, graphical Lasso
- Support Vector Machines
- Regression with non-convex penalties